COS 214 Project Report

Group Name: sudo rm -rf / --no-preserve-root

COS 214 Report

Table of Contents

Research Brief

Most of the research for implementing the project and which features it should include came from the *Final Project Specification* (Department of Computer Science). Any other ideas came from discussions among the group to implement certain features in a certain way.

Plant Care Research

The idea for plants requiring water, and having a health system that can be repaired with fertilizer, comes from the *Final Project Specification*, specifically the section titled “Greenhouse/Garden Area”. The plant state life cycle, and plant care features come directly from this section of the document.

Plant Types

The plant types were decided arbitrary and decided on common knowledge, such a cactus requiring low water, etc. The variables on the plants are important to running a nursery, as they are the product being sold towards customers, but to make the system extensible, the plants only store and set variables, and any functionality is delegated to other classes.

Staff

The staff are handled by a chain of responsibility system that receives commands that can either be handled by the handler, or passed onto another staff. The idea for this system comes directly from *Practical 4* (Department of Computer Science), with how the tickets were handled with a combination of command and chain of responsibility. This allows the system to be expandable by adding new handlers, and new commands separately from each other, which would ease development, and allow these features to be coupled later.

Game Loop

The Game Loop was decided in one of our first meetings. We decided to go for a game loop as we thought it was a unique idea, and gave a concrete basis for the project to be built off. This meant we needed to have some system to save progress, and we decided to serialize our data that needed to be remembered with a json format using the nlohmann library (Lohmann). We then went from previous experience to decide on what should be done in the game loop.

Customers

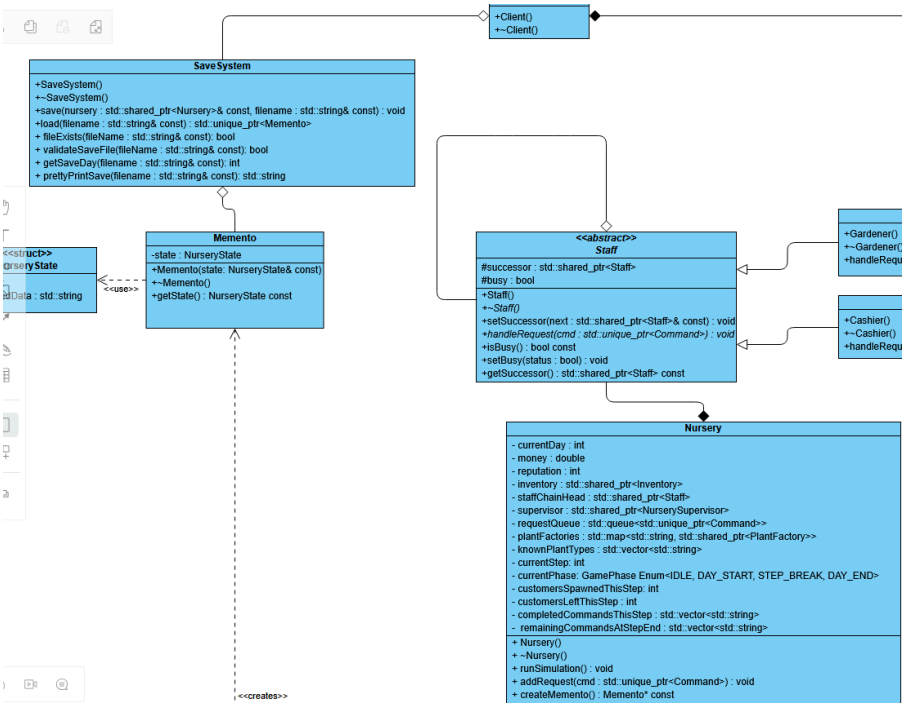
Due to deciding on structuring the project as a game, the Customers serve an integral role in the system, as they are the reason that the game can progress at all. For this reason the customer order deciding logic is quite complicated, using a builder to add decorators and plants to their order, and either being satisfied with their order or not.

Design Patterns

Please note that the relevant UML diagrams might have arrows coming in and out that are not shown in the image, as the pattern is part of a whole.

1. Memento

UML



Location

The Memento design pattern is implemented in the Save Architecture. This links to Functional Requirement 12.

Participants

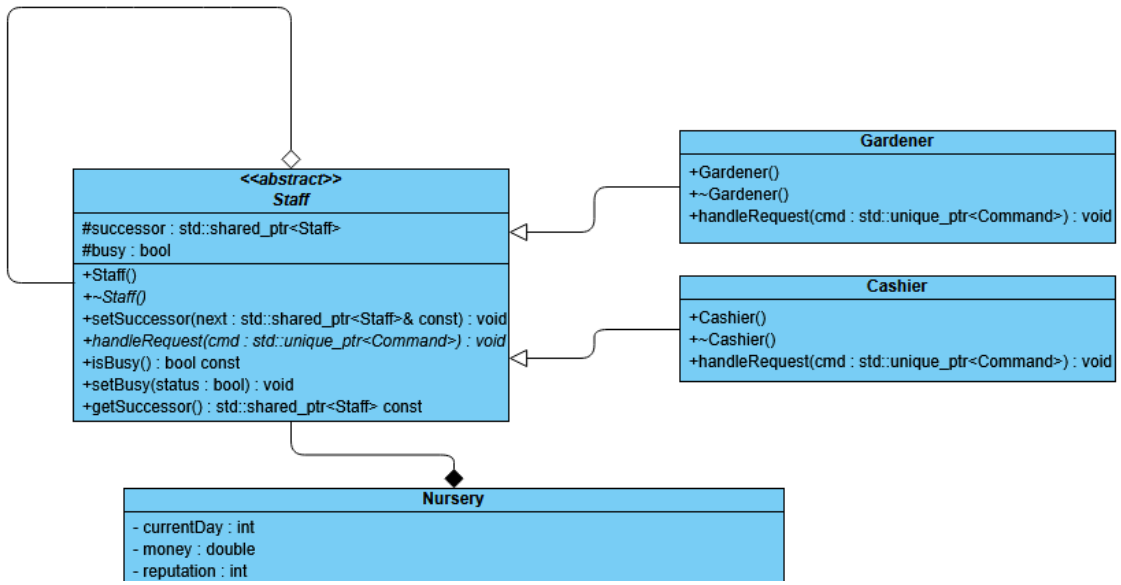
- Client - Client
- Originator - Nursery
- Memento - Memento
- Caretaker - SaveSystem

Reasoning

Memento was implemented to allow the current nursery state to be saved to a json file, which can then be input into the caretaker at runtime to restore the state to a previous state from a previous runtime. This allows for the game to have persistent data, and not require to start from scratch.

2. Chain of Responsibility

UML



Location

The Chain of Responsibility design pattern is implemented in the Staff Architecture. This links to Functional Requirement 10.

Participants

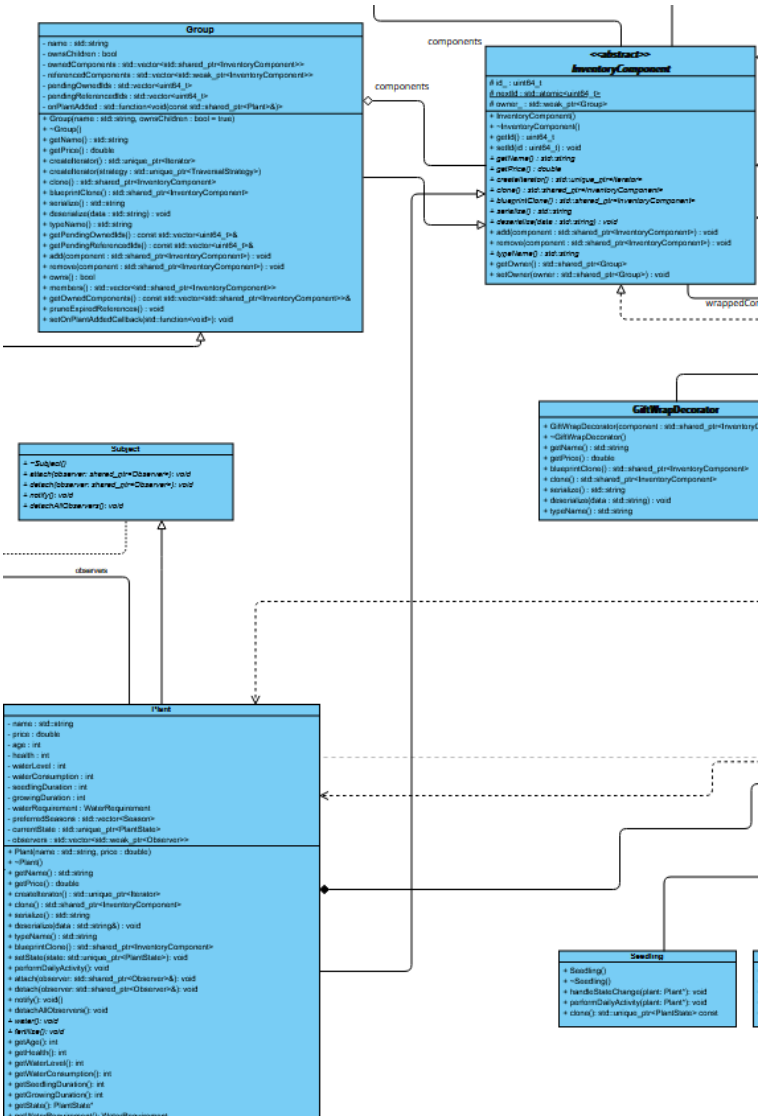
- Handler - Staff
- ConcreteHandler - Gardener/Cashier
- Client - Nursery

Reasoning

Chain of responsibility was decided to handle the staff system, such that a problem can be issued independently from the object that will respond to the issue, which allows the user to independently create these issues, and they will automatically get assigned to the correct handler. It allows new staff types to be added to the system easily as new issues that cannot be handled by the current staff get discovered. It also decouples the solving of issues from the issuing of requests.

3. Composite

UML



Location

The Composite design pattern is implemented in the Inventory Architecture. This links to Functional Requirement 1.

Participants

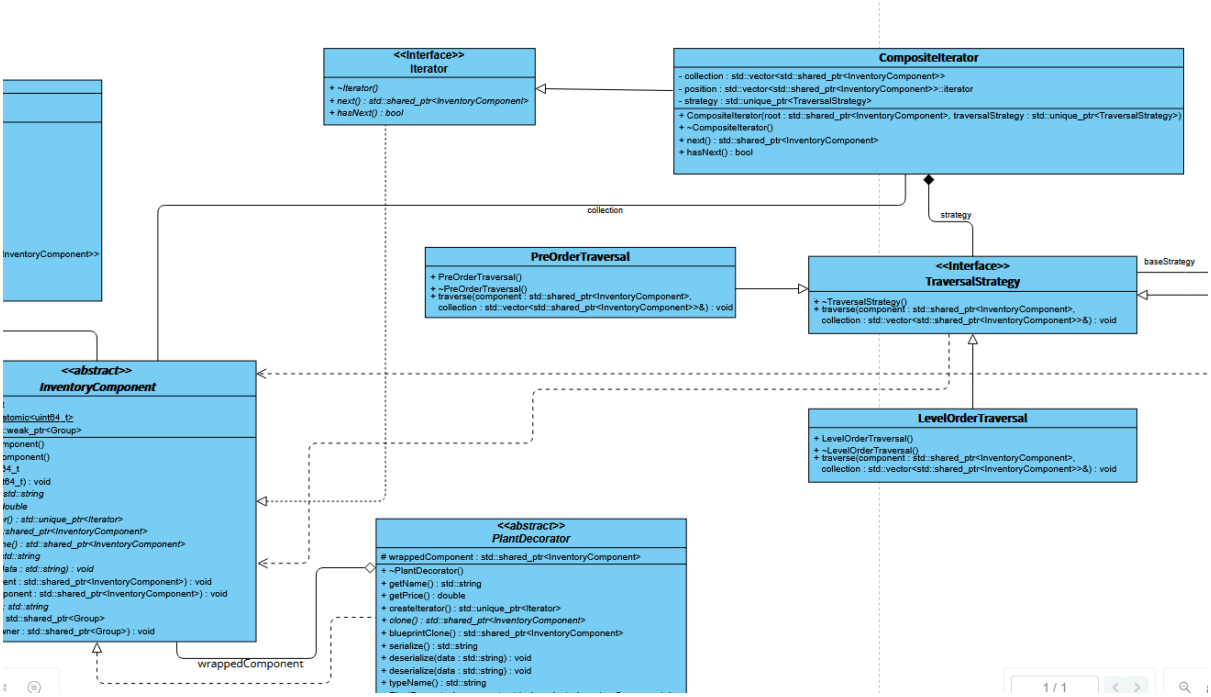
- Component - InventoryComponent
- Composite - Group
- Leaf - Plant

Reasoning

The composite design pattern was chosen to manage the inventory, so that all plants can be accessed from one location, and so that plants could have implicit relationships with each other by their parents. This is achieved by having plants be the leaves of the composite, and groups which establish certain properties be the composites to which leaves belong. This allows plants to easily be added and removed to these groups, while not needing to know the exact structure of how all the data is stored.

4. Iterator

UML



Location

The iterator pattern is implemented in the Inventory Architecture. This links to Functional Requirement 5

Participants

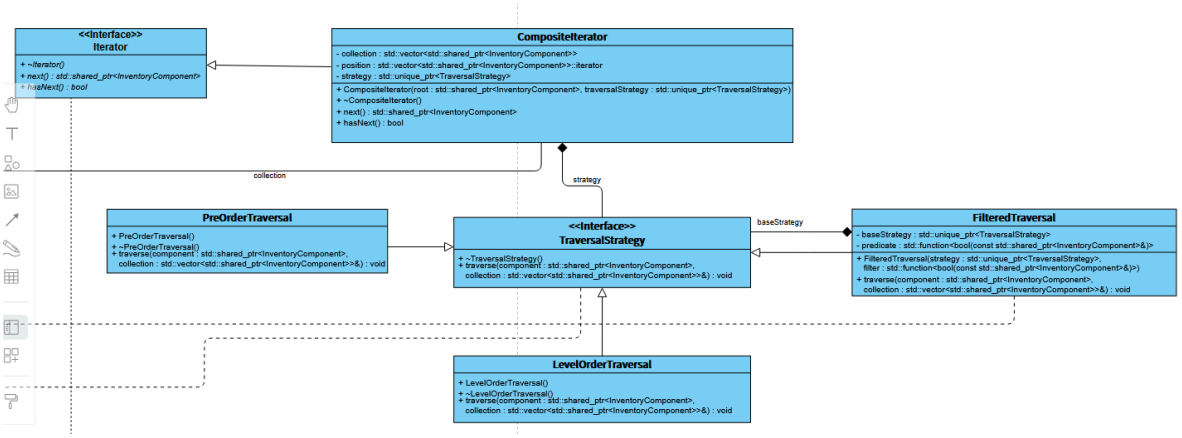
- Iterator - Iterator
- ConcreteIterator - CompositeIterator
- Aggregate - InventoryComponent
- ConcreteAggregate - Group

Reasoning

The Iterator design pattern was added as there is no standard way to use STL iterators to iterate through a composite, and a standardised way to navigate through the inventory was required. An external iterator was used to allow the client to decide what to do with the current element.

5. Strategy

UML



Location

The Strategy pattern is implemented in the Inventory Architecture, more specifically the Iterator. This links to Functional Requirement 6

Participants

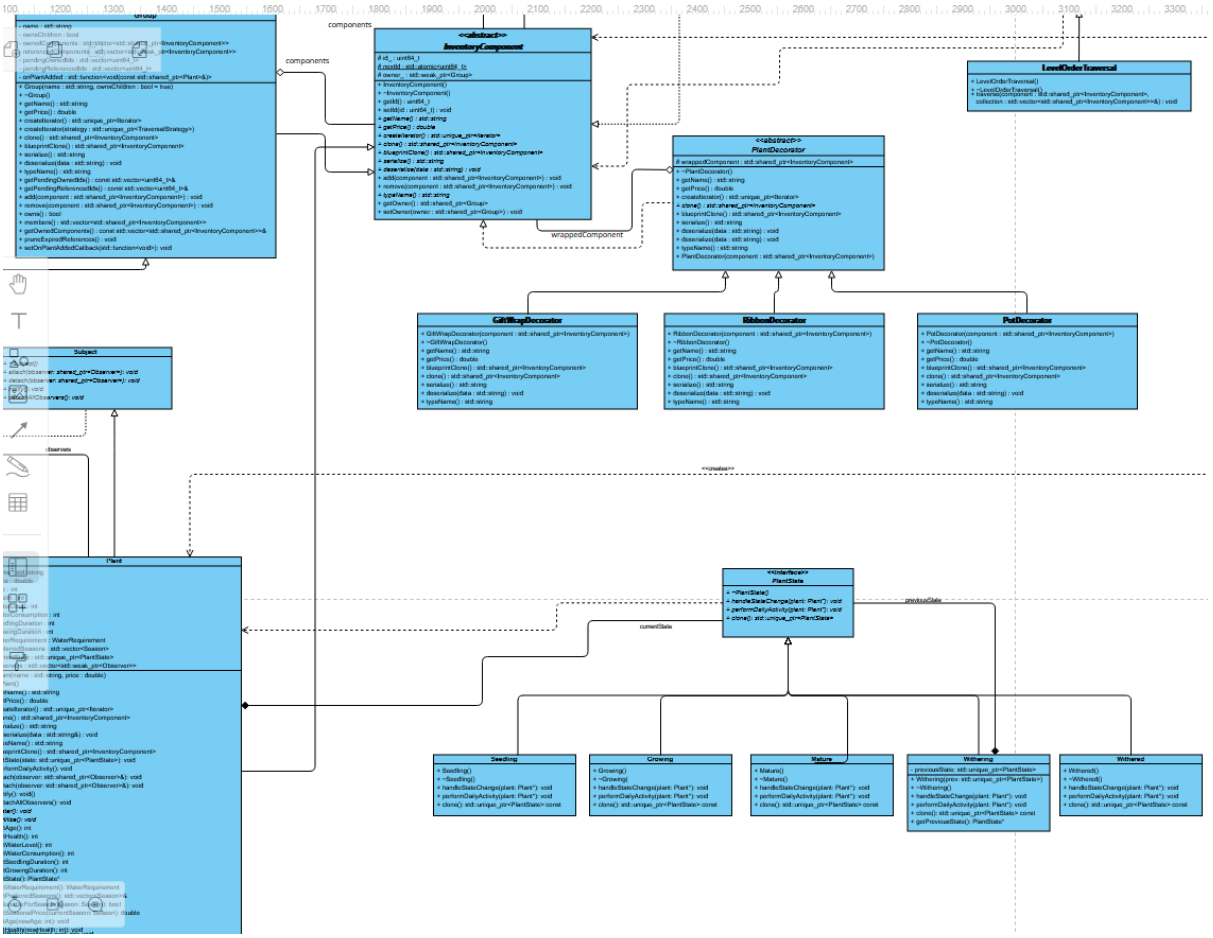
- Context - CompositeIterator
- Strategy - TraversalStrategy
- ConcreteStrategy - PreOrderTraversal/LevelOrderTraversal/FilteredTraversal

Reasoning

The strategy pattern was implemented inside of the iterator to allow the inventory to only require 1 iterator, and this iterator can switch which pattern it is using depending on what the client needs to retrieve, allowing the iterator to be much more flexible.

6. Decorator

UML



Location

The Decorator pattern is implemented in the Inventory Architecture. This links to Functional Requirement 2

Participants

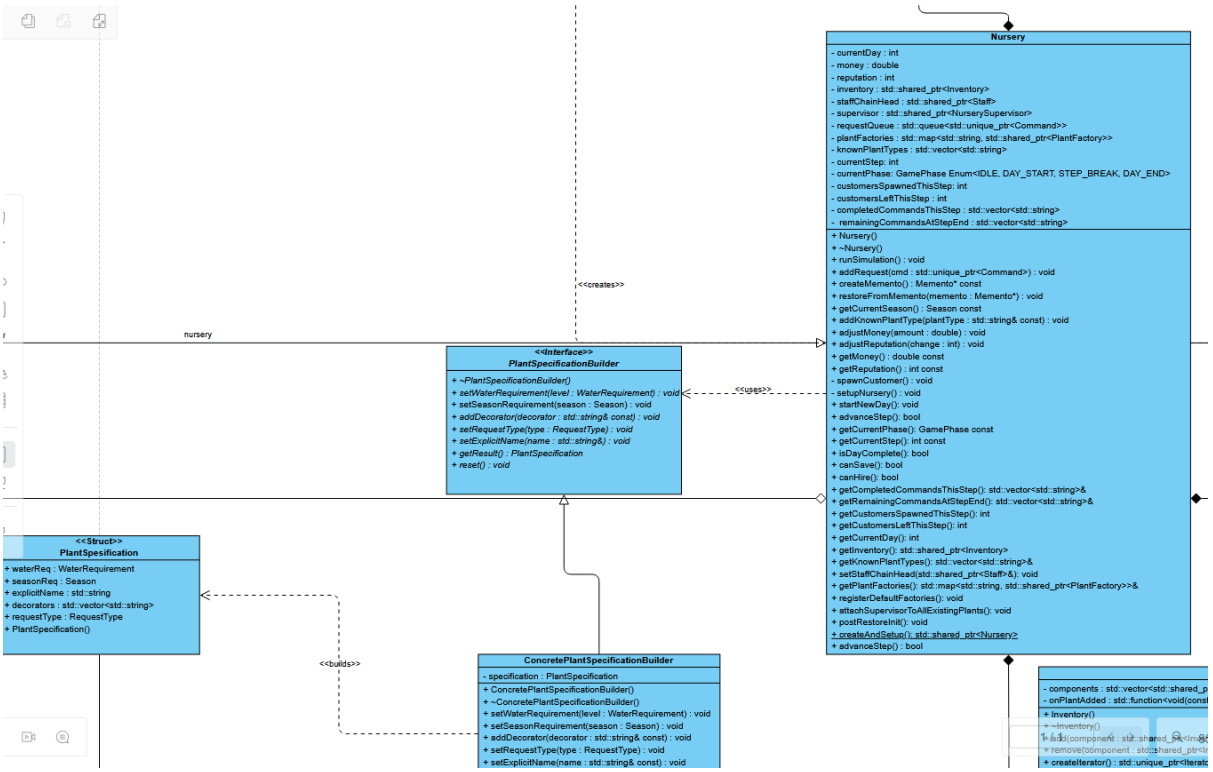
- Component - InventoryComponent
- Decorator - PlantDecorator
- ConcreteDecorator - GiftWrapDecorator/RibbonDecorator/PotDecorator
- ConcreteComponent - Plant

Reasoning

The decorator pattern was added so that customers are able to add additional external features to their plants, without needing to modify the plant they are ordering. This also allows additional decorators to be added in future to satisfy customer commands.

7. Builder

UML



Location

The Builder pattern is part of the order architecture. This links to Functional Requirement 11.

Participants

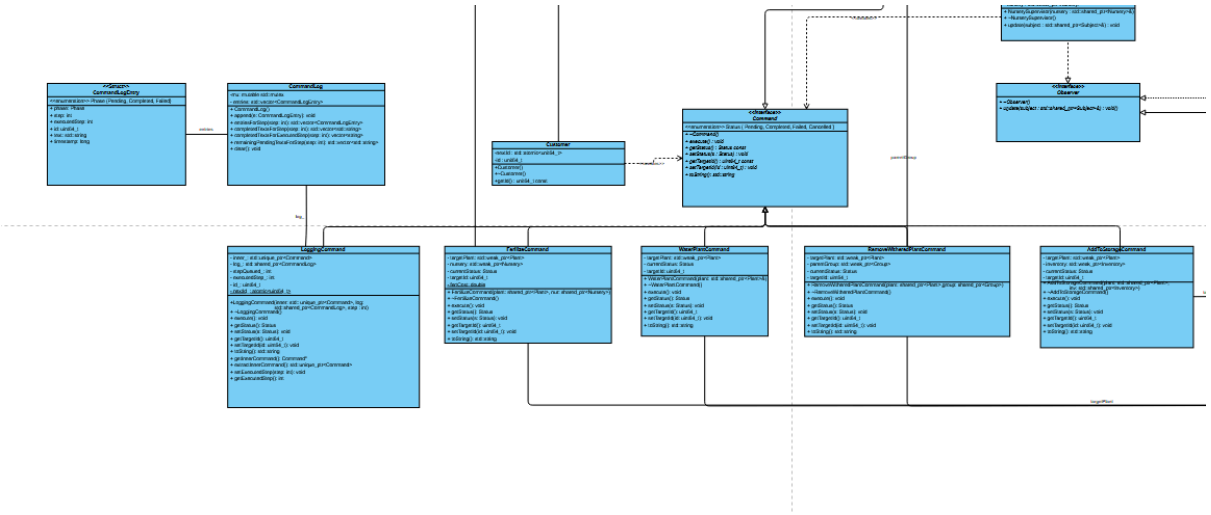
- Product - PlantSpecification
- ConcreteBuilder - ConcretePlantSpecificationBuilder
- Builder - PlantSpecificationBuilder
- Director - Nursery

Reasoning

The Builder design pattern was used, since customer orders can be very complex, requiring decorators, or which plant is being built, or even if the plant they are ordering is non-determinate, and instead the customer asks for a recommendation that follows a set of criteria. The builder pattern allows the orders to be constructed in a step-by-step manner with a single instruction from the client. This allows the system the flexibility to randomise orders which customers have.

8. Command

UML



Location

Command is implemented in the Nursery architecture, and then passed to the correct architecture. This links to Functional Requirement 9.

Participants

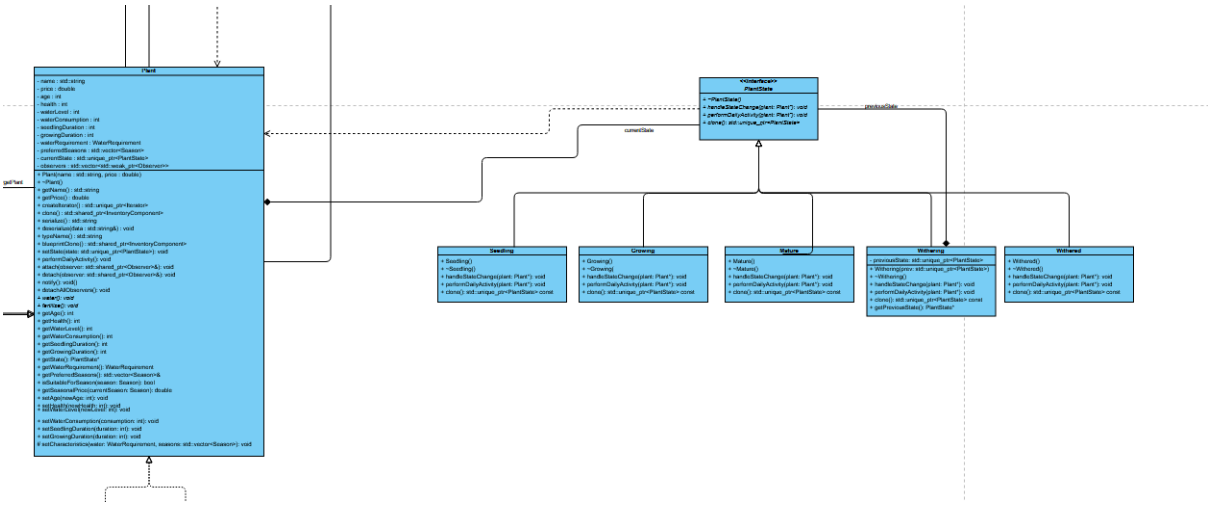
- Command - Command
- ConcreteCommand - FertilizeCommand/LoggingCommand/RemoveWitheredPlantCommand/ WaterPlantCommand/FulfillCustomerCommand
- Invoker - NurserySupervisor/Customer
- Receiver - Plant/Nursery
- Client - Nursery

Reasoning

The command pattern is used so that issues can be created without needing to be immediately solved. This allows the game loop to work, as all actions for that step can be put in the queue, which can then be executed later. It also allows new actions to be added to the system relatively easily, as long as the appropriate handler is able to understand the content of the new command object, meaning they are loosely coupled.

9. State

UML



Location

The State pattern is implemented in the Plant architecture. This links to Functional Requirement 7.

Participants

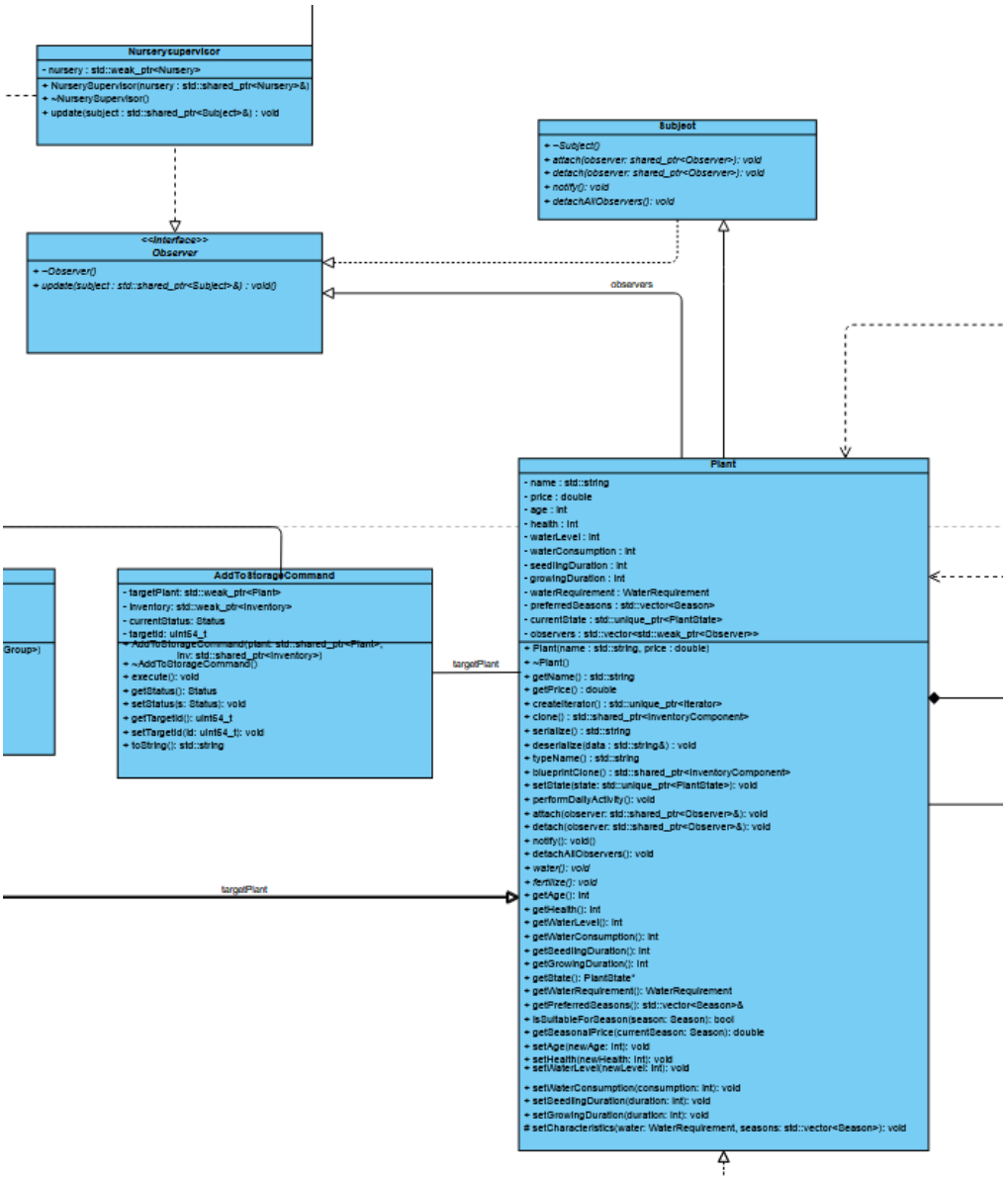
- Context - Plant
- State - PlantState
- ConcreteState - Seedling/Growing/Mature/Withering/Withered

Reasoning

The State pattern was applied so that plants can enter different stages of their lifecycle, while still having the same interface to interact with the plants. State switching is handled internally by the states, and is not intended to be switched from the outside. These states allow final states in Mature and Withered to be stubbed.

10. Observer

UML



Location

The observer pattern is implemented in the Nursery Architecture. This links to Functional Requirement 8

Participants

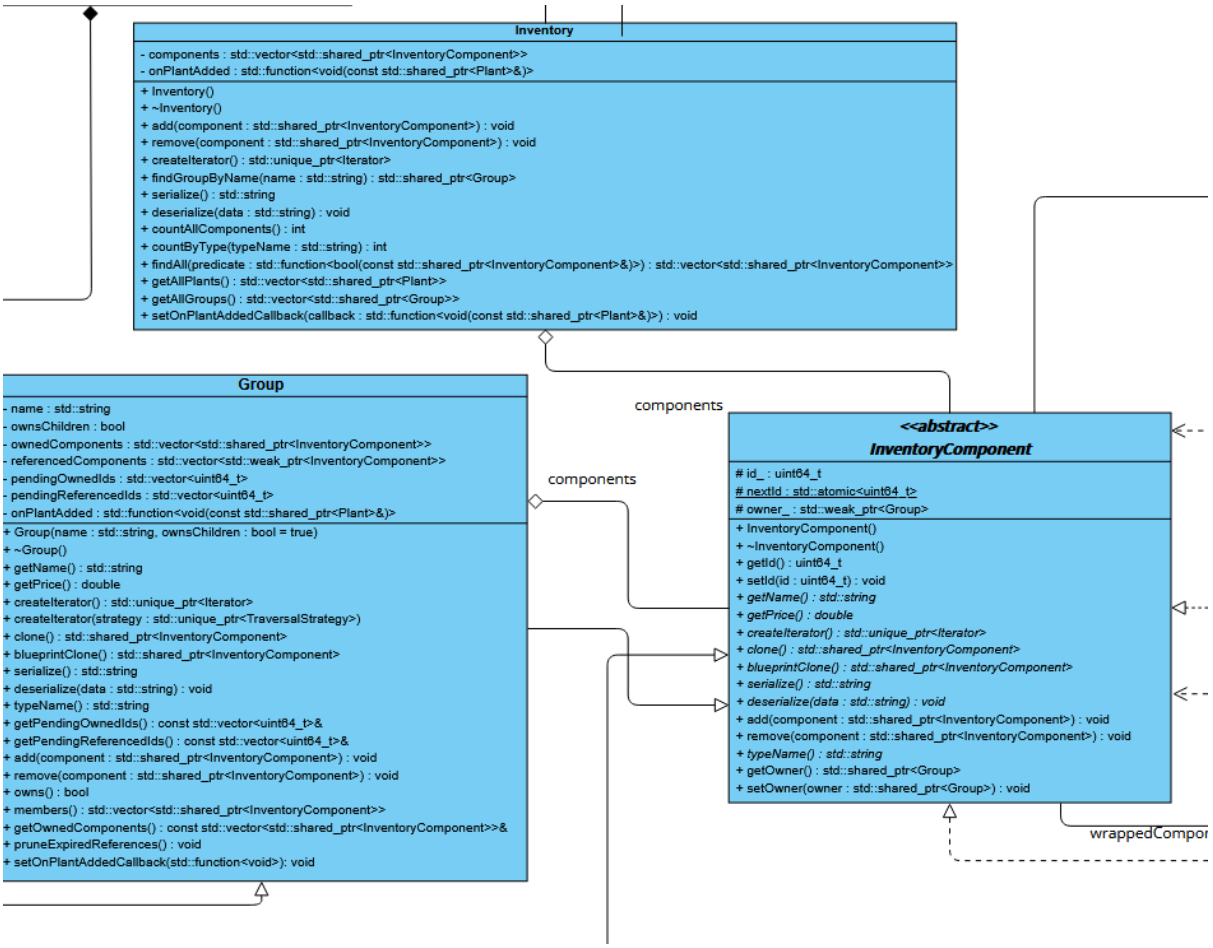
- Subject - Subject
- Observer - Observer
- ConcreteObserver - NurserySupervisor
- ConcreteSubject - Plant

Reasoning

The observer pattern was decided so that commands to care for plants can be created by the system automatically, and so that whenever a plant's state changes, the nursery is aware of the change and can retrieve the data from the plant.

11. Prototype

UML



Location

The prototype is implemented in the Inventory Architecture. This links to Functional Requirement 3.

Participants

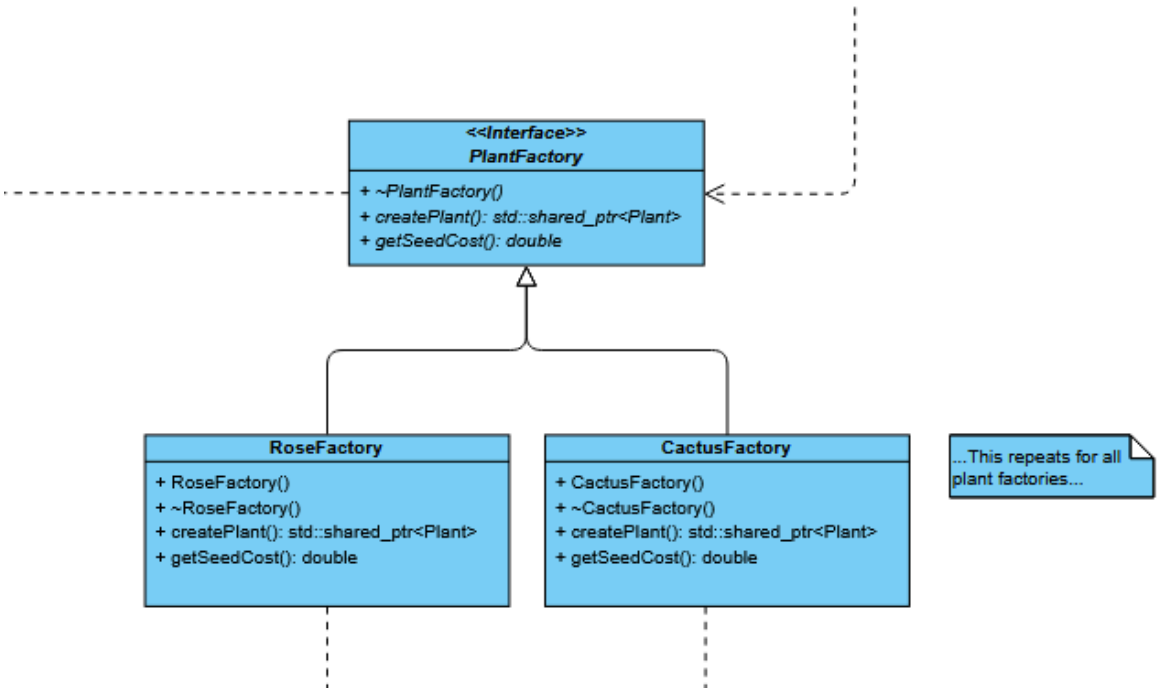
- Prototype - InventoryComponent
- ConcretePrototype - Group
- Client - Nursery

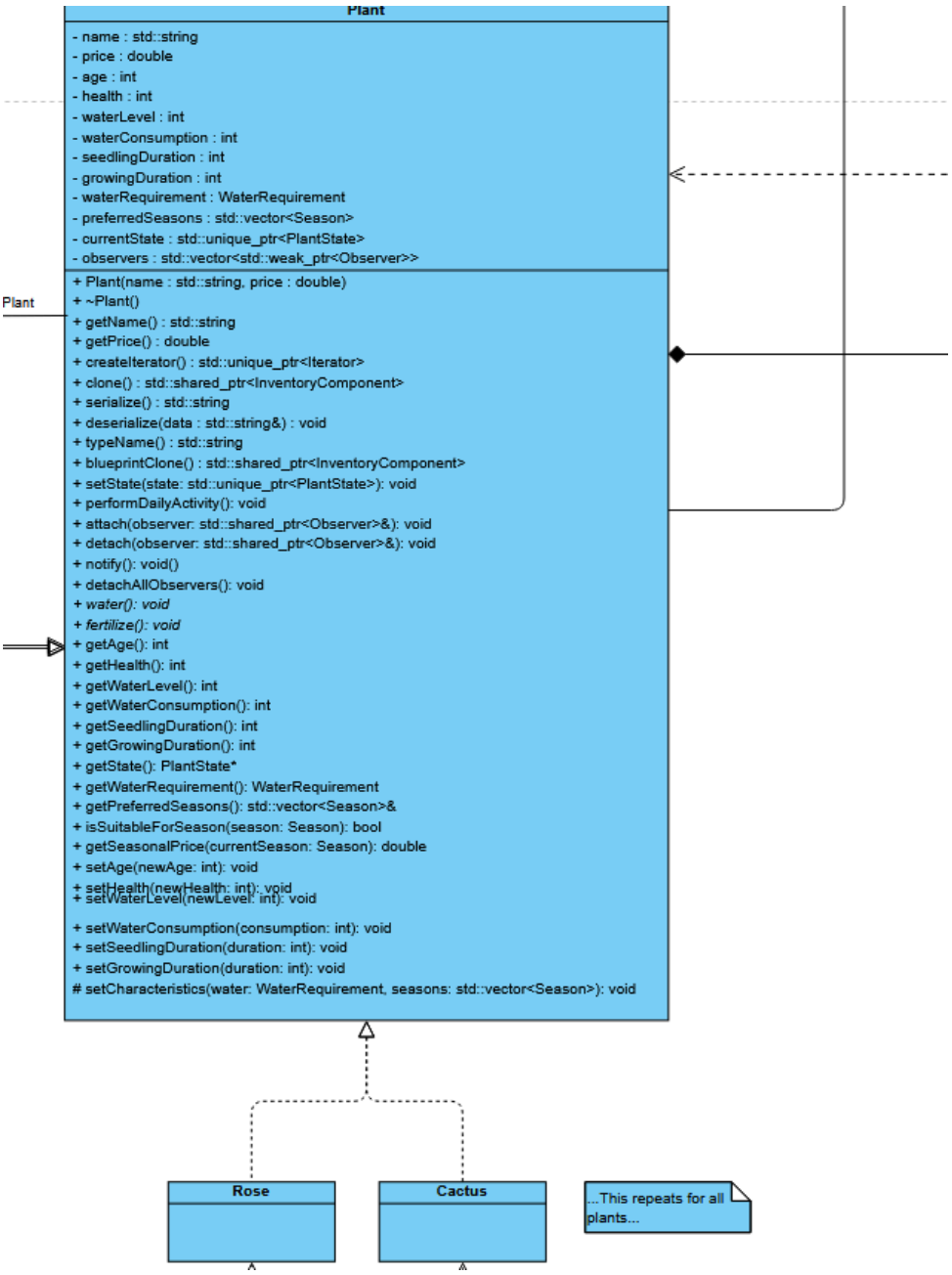
Reasoning

The prototype pattern was used since the cost of instantiating a group of plants to be added somewhere else is expensive, so instead an existing group can be cloned and modified if need be.

12. Factory Method

UML





Location

The Factory Method is part of the Nursery Architecture. This links to Functional Requirement 4.

Participants

- Creator - PlantFactory
- ConcreteCreator - RoseFactory/CactusFactory/….
- Product - Plant
- ConcreteProduct - Rose/Cactus/….

Reasoning

The factory method was decided to be used to create new leaves for the composite inventory. This allows the creation of plants to be abstracted and created through a single method, without having to worry about how the factory creates the plant.

Functional and Non-Functional Requirements

Functional Requirements

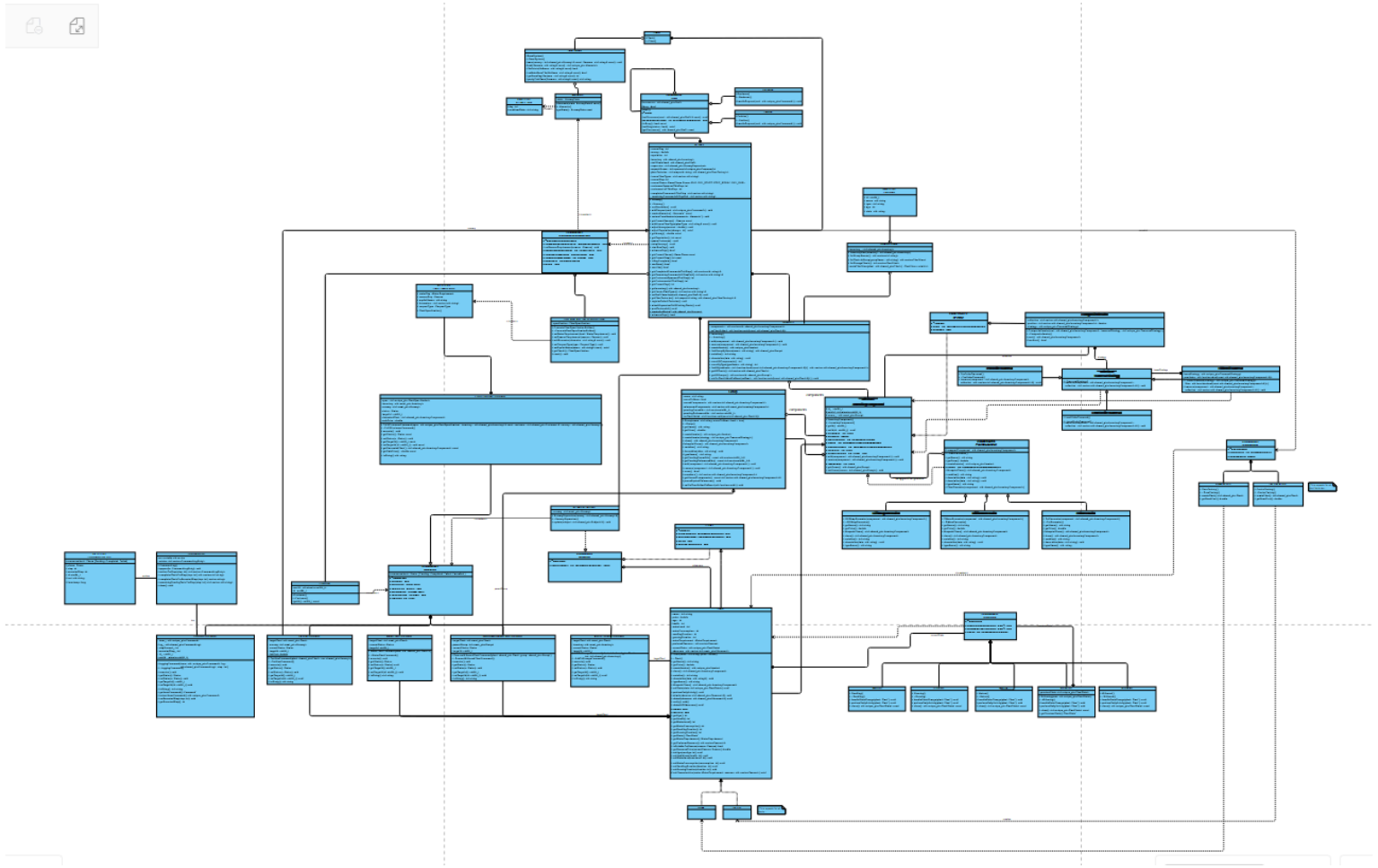
1. The system shall manage inventory items, whether they are individual plants or groups of plants (plots), using a common interface for operations like listing contents or calculating value.
2. The system shall allow for the dynamic addition of customizations, such as decorative pots or gift wrapping, to any plant without changing the plant's fundamental class.
3. The system shall allow the user to create an exact, independent copy of a complex arrangement of plants within a group to replicate it elsewhere.
4. The system shall allow for the addition of new, unique plant types without requiring modification to the core nursery logic that stocks the inventory.
5. The system shall provide a standardized way to traverse all plants in the inventory, regardless of how they are structured in groups or sub-groups.
6. The traversal of the inventory shall support different algorithms (e.g., by plot, by mature plants) that can be selected at runtime depending on the task.
7. A plant's behaviour and properties (e.g. fertilizer consumption) shall change automatically as it progresses through different lifecycle stages (e.g., seedling, growing, mature).
8. The system shall automatically detect when a plant's internal state requires attention (e.g., needs watering) and generate a corresponding work request without the plant being aware of the staff or task system.
9. All tasks, such as watering a plant or fulfilling a customer order, shall be encapsulated as distinct, self-contained command objects that can be placed on a central queue.
10. The system shall route tasks from the central queue to the appropriate staff member (e.g., Gardener, Cashier) based on their role and capabilities, allowing the task to be passed on if a staff member cannot handle it.
11. The system shall construct complex and varied customer requests by dynamically combining multiple attributes (e.g., water needs, sun needs, desired name) in a step-by-step process.
12. The system shall be able to save the entire state of the nursery (inventory, staff, day, etc.) to a persistent format and restore it at a later time to resume the simulation.

Non-functional Requirements

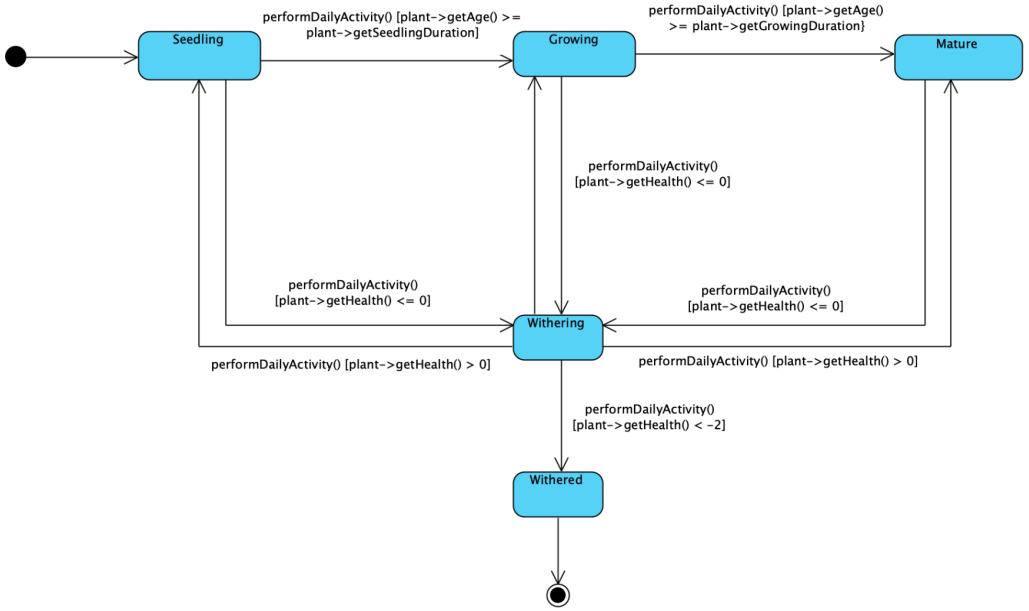
1. The system will be designed to allow the addition of new plant types, staff roles, or customer request types with minimal changes to existing, unrelated code.
2. The system's architecture will be highly modular, with a clear separation of concerns. The logic for a plant's lifecycle must be fully decoupled from the task management system, and the creation of tasks must be fully decoupled from their execution.
3. The system's save and load functionality will reliably persist and restore the game state. A restored session must be identical to the state of the simulation at the moment it was saved, with no loss of data regarding inventory, plant states, or queued tasks.

Diagrams

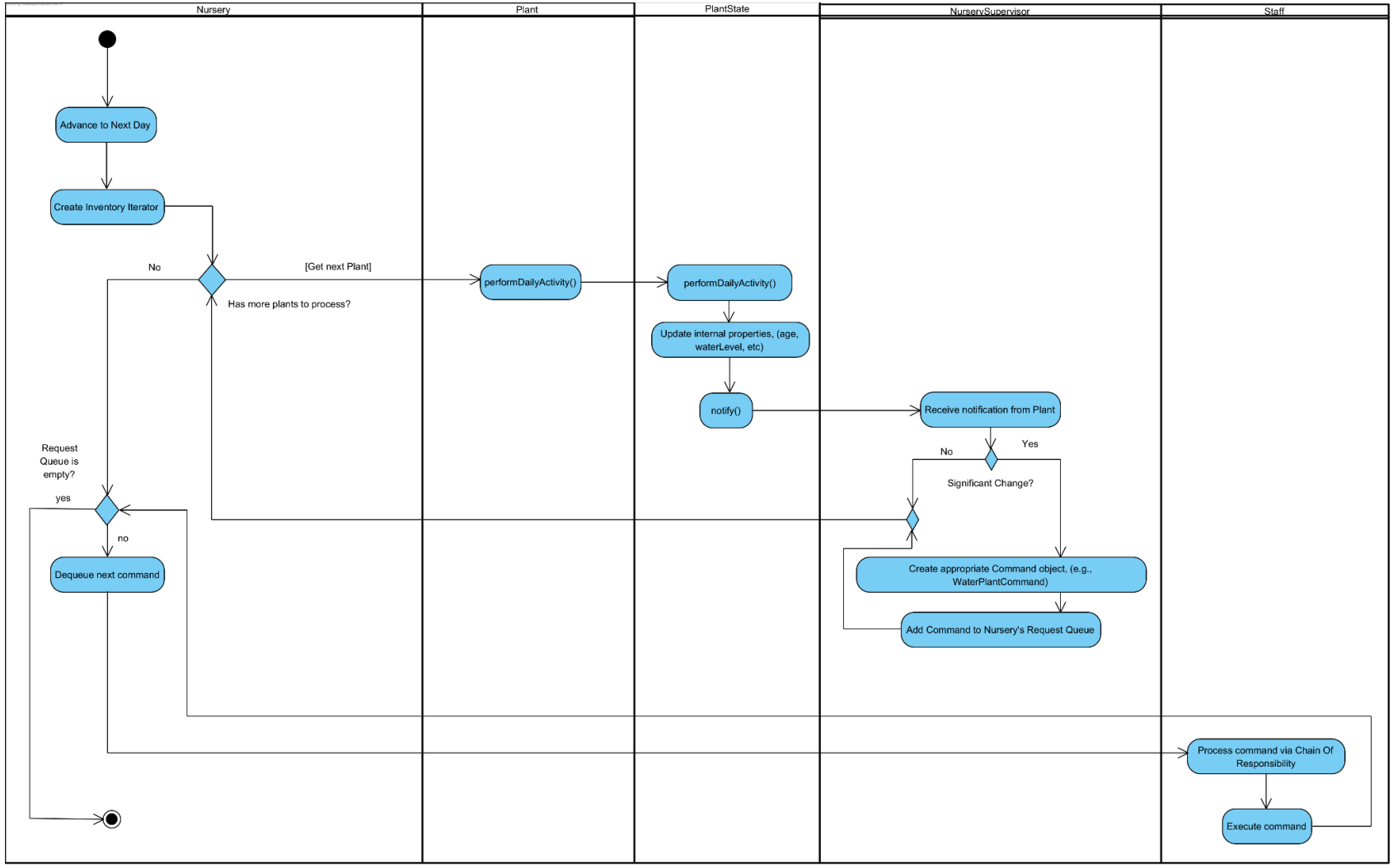
Class



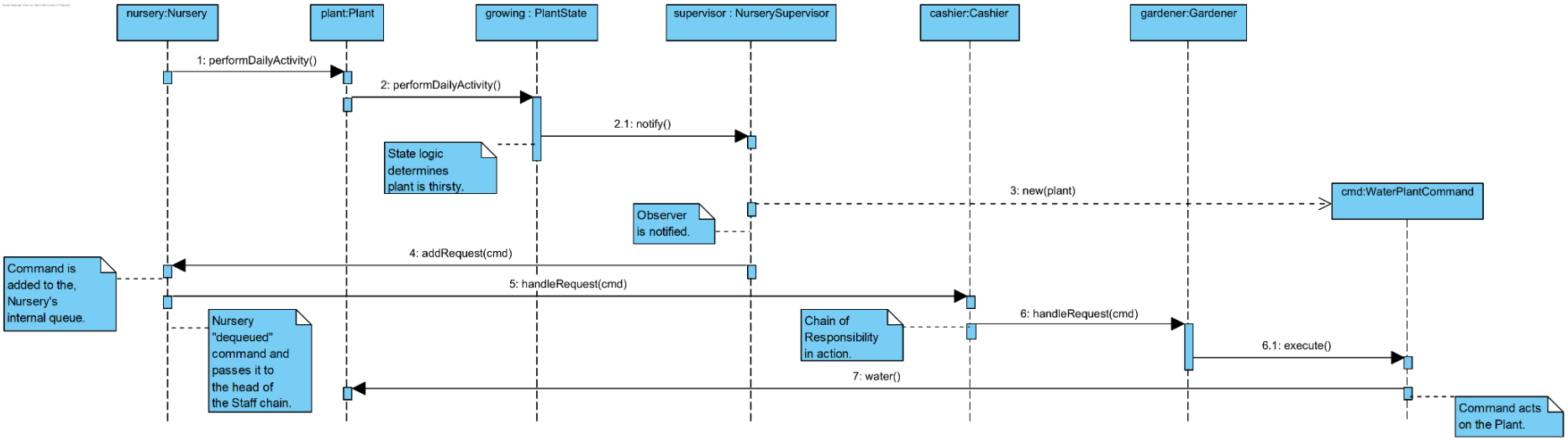
State


Seedling
Growing
Mature
Withering
Withered
performDailyActivity() [plant->getAge() >= plant->getSeedlingDuration]
performDailyActivity() [plant->getAge() >= plant->getGrowingDuration}
performDailyActivity() [plant->getHealth() <= 0]
performDailyActivity() [plant->getHealth() <= 0]
performDailyActivity() [plant->getHealth() <= 0]
performDailyActivity() [plant->getHealth() > 0]
performDailyActivity() [plant->getHealth() > 0]
performDailyActivity() [plant->getHealth() < -2]

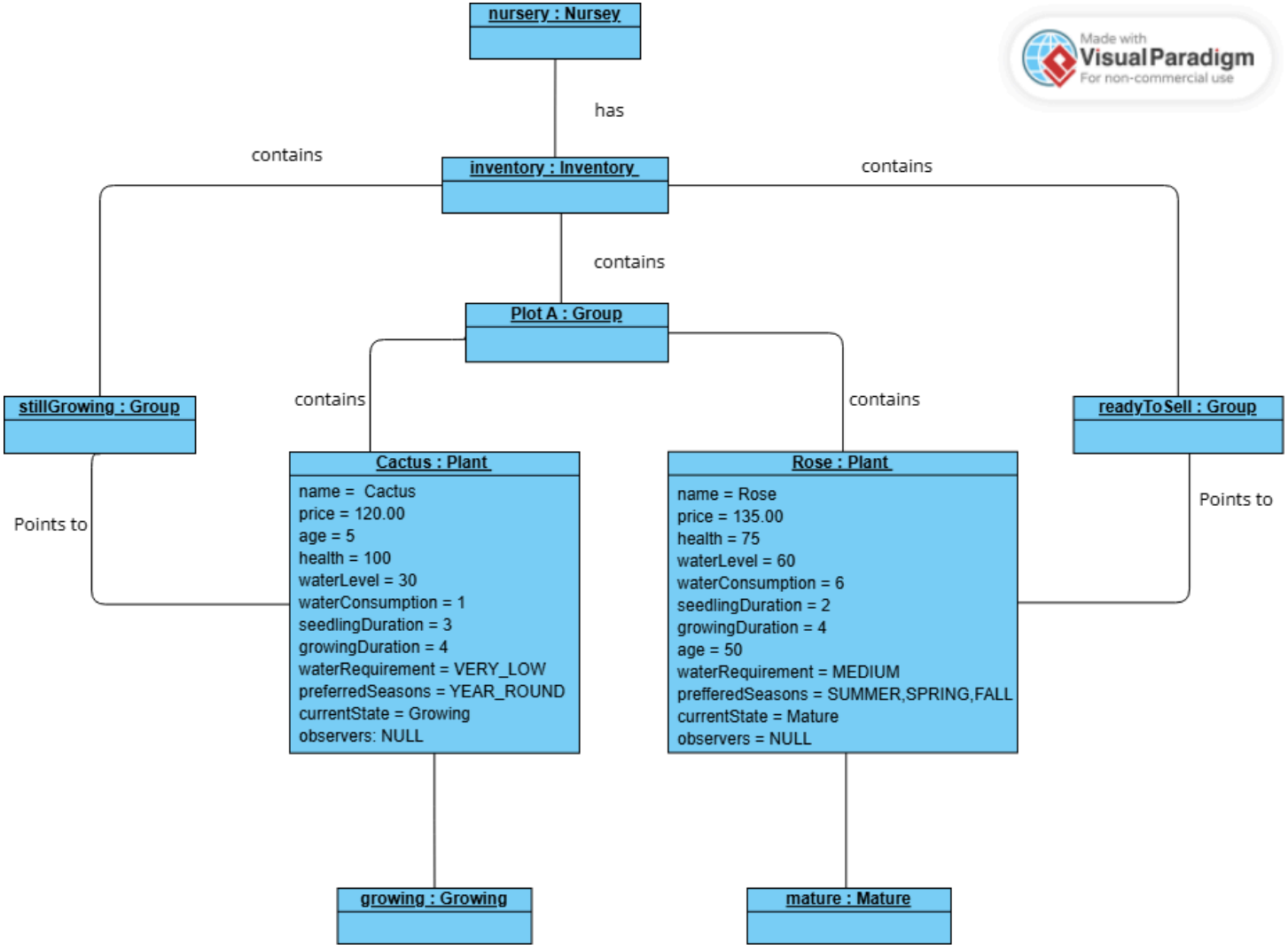
Activity


Nursery
Plant
PlantState
NurserySupervisor
Staff
Advance to Next Day
Create Inventory Iterator
No
[Get next Plant]
Has more plants to process?
performDailyActivity()
performDailyActivity()
Update internal properties, (age, waterLevel, etc)
notify()
Receive notification from Plant
No
Yes
Significant Change?
Request Queue is empty?
yes
no
Dequeue next command
Create appropriate Command object, (e.g., WaterPlantCommand)
Add Command to Nursery's Request Queue
Process command via Chain Of Responsibility
Execute command

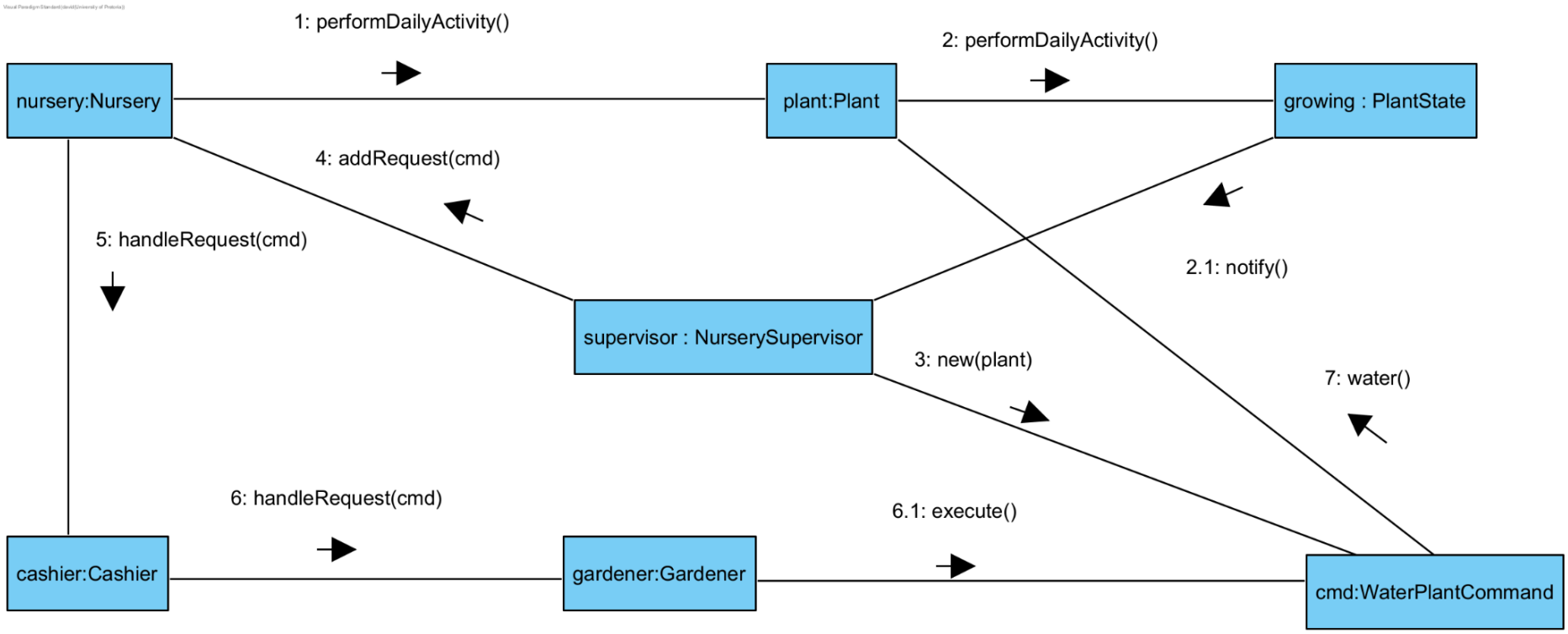
Sequence


nursery:Nursery
plant:Plant
growing : PlantState
supervisor : NurserySupervisor
cashier:Cashier
gardener:Gardener
cmd:WaterPlantCommand
1: performDailyActivity()
2: performDailyActivity()
2.1: notify()
State logic determines plant is thirsty.
Observer is notified.
3: new(plant)
4: addRequest(cmd)
Command is added to the, Nursery's internal queue.
5: handleRequest(cmd)
Nursery "dequeued" command and passes it to the head of the Staff chain.
Chain of Responsibility in action.
6: handleRequest(cmd)
6.1: execute()
7: water()
Command acts on the Plant.

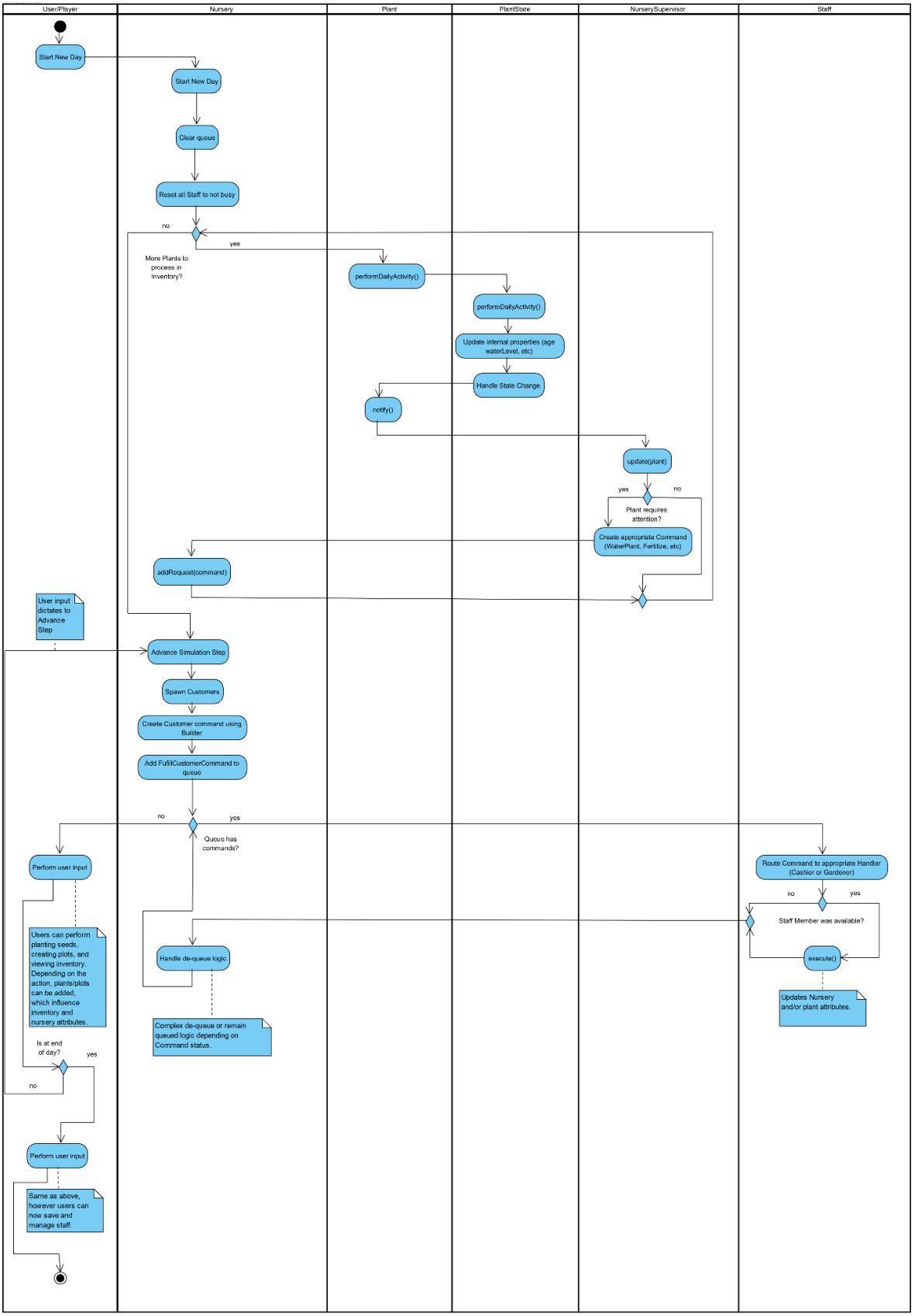
Object


nursery : Nursey
has
contains
inventory : Inventory
contains
contains
Plot A : Group
contains
contains
stillGrowing : Group
readyToSell : Group
Points to
Points to
Cactus : Plant
name = Cactus
price = 120.00
age = 5
health = 100
waterLevel = 30
waterConsumption = 1
seedlingDuration = 3
growingDuration = 4
waterRequirement = VERY_LOW
preferredSeasons = YEAR_ROUND
currentState = Growing
observers: NULL
Rose : Plant
name = Rose
price = 135.00
health = 75
waterLevel = 60
waterConsumption = 6
seedlingDuration = 2
growingDuration = 4
age = 50
waterRequirement = MEDIUM
prefferedSeasons = SUMMER,SPRING,FALL
currentState = Mature
observers = NULL
growing : Growing
mature : Mature
Made with VisualParadigm
For non-commercial use

Communication



High Level Activity



Miscellaneous

UI (Cpp-Terminal)

For the user interface, due to time constraints we decided to make the GUI inside of the terminal with a library called cpp-terminal (Link to which can be found as a submodule in the repository). To link the library, we had to make our makefile pre-compile the library, since the library uses cmake, and tell cmake to generate a static library that could be linked to our program at compile time.

Cpp-terminal allows the terminal to be interactive, accepting button input and menu design, which was required, as otherwise showcasing the game would be impossible.

Later in development it was discovered that this library has a memory leak with valgrind, but the memory leak only occurs when the program closes due to a thread not joining back to the main thread properly, and the memory leak should not occur during normal execution, only when the program is ending.

Automated Testing

For automated testing we used doctest to test the program locally, and we integrated this testing into the github repository with github actions, which will run the test cases whenever a pull request is created.

Github

For our github usages, we ended up using a branching strategy that involves branching off of a main developer branch, and then using the github issues and project board feature to describe which parts of the project had to be done, and people could then assign themselves to these issues. For merging we used pull requests and merging to merge the changes together, with either Stephan or David reviewing the code of every pull request to determine if it was safe to merge, and met the issue specifications. As stated earlier there was also automated linting on pull requests to ensure that all test cases passed.

References

Department of Computer Science. *Final Project Specification*. 25 September 2025. *Clickup*, University of Pretoria, https://clickup.up.ac.za/ultra/courses/_177686_1/outline/file/_4595899_1. Accessed 31 October 2025.

Department of Computer Science. *Practical 4 Specifications*. 3 October 2025. *Fitchfork*, University of Pretoria, https://ff.cs.up.ac.za/assignments/604/. Accessed 9 October 2025.

Lohmann, Niels. “JSON for Modern C++.” *JSON for Modern C++*, 2013, https://json.nlohmann.me/. Accessed 31 October 2025.